

Food Insecurity Forecast Tool

— Retrain & Refresh Guide

How to refit the persisted 3-year-lag ridge model's weights against a new year of Map the Meal Gap and ALICE data — only that one spec, ridge only. This is the tool that changes `models/cc/lag3_ridge.json`; if you just want to score data with what's already saved there, you don't need this guide.

Assumes the one-time setup (Python, packages, licensed data access) from the [Setup & Usage Guide](#) is already done — this guide only covers what's different about retraining.

When should I actually do this?

Not every time you score. Retrain when a genuinely new year of MMG data has landed and you want the model's coefficients to reflect it — the *Model Review* document's own recommendation is to revisit once each additional year arrives, since the 3-year lag spec is trained on very few usable transition years to begin with. Scoring with an older but still-valid model is completely fine in between.

QUICK REFERENCE — FOR ANYONE WHO'S ALREADY SET UP

```
python scripts/retrain_lag3_model.py \  
  --mmg "path\to\MMG_2019-2023.xlsx" "path\to\MMG_2024.xlsx" \  
  --alice "path\to\alice_county.csv"
```

This overwrites `models/cc/lag3_ridge.json` — the same file the scoring tool reads by default — and appends a row to `results/cc/lag3_retrain_history.csv`. Both flags shown are required.

1 What this needs that scoring doesn't

Scoring a year only needs one snapshot. Fitting the 3-year lag needs the model to see pairs of years that are 3 years apart for the same ZIP — so `--mmg` takes one *or more* workbooks, passed

in chronological order, and together they need to span enough years to form at least one such pair (e.g. 2020 and 2023 data, to learn the 2020 → 2023 relationship). The repository's own `data/external/` already has the two files used to train the current production model:

FILE	COVERS
<code>MMG_2019-2023.xlsx</code>	2020–2023 (multi-year release)
<code>MMG_2024.xlsx</code>	2024 (single-year release)

If rows for the same ZIP and year show up in more than one file you pass, the first file's value wins — another reason to list them oldest-first.

2 Running the tool

Required flags

FLAG	WHAT TO PASS
<code>--mmg "path.xlsx"</code> <code>["path2.xlsx" ...]</code>	One or more raw MMG workbooks, oldest first, spanning enough years to form a 3-year pair.
<code>--alice "path.csv"</code>	The Florida ALICE county CSV. No default — always pass the current file explicitly.

Optional flags

FLAG	WHEN YOU NEED IT
<code>--output "path.json"</code>	Write somewhere other than <code>models/cc/lag3_ridge.json</code> , to inspect a candidate fit (e.g. score against it with the scoring tool's <code>--model</code> flag) before deciding to promote it.
<code>--results-log "path.csv"</code>	Log to somewhere other than <code>results/cc/lag3_retrain_history.csv</code> .
<code>--state ""</code>	Skip the Florida-specific accuracy slice and evaluate nationally only.
<code>--food-bank ""</code>	Skip the Feeding Tampa Bay-specific accuracy slice.
<code>--alpha-grid "0.01,0.1,0.5,1,2"</code>	Search a different set of ridge penalty strengths instead of the built-in default grid.

3 What happens when you run it

1. Reads and validates every `--mmg` file and the ALICE file (right sheets, right columns).
2. Builds the same ZIP-county-year panel the scoring tool builds, so a refit model and the data it's later scored against are always constructed identically.
3. Checks there's enough paired 3-years-apart data to fit against — fails with a specific message rather than a raw traceback if not.
4. Fits the 3-year lag ridge model, choosing its penalty strength (`alpha`) and validation method automatically (see below).
5. Saves the fitted weights to `--output` (`models/cc/lag3_ridge.json` by default).
6. Reloads what it just saved and re-checks its predictions match the in-memory fit exactly — refuses to leave a broken file in place if they don't.
7. Appends one summary row to the results log.

4 Trusting the fit

Printed near the top of the run, `Validation method` tells you how seriously to take the reported accuracy numbers:

METHOD	MEANING
<code>time_holdout</code> TRUSTWORTHY	The model trained on earlier year-pairs and was tested on the most recent one it never saw — a real, honest holdout. This is what the current production model uses.
<code>5_fold_cv_fallback</code> LOWER CONFIDENCE	Your <code>--mmg</code> files only produced <i>one</i> usable 3-years-apart transition, so there was no separate year left to hold out — the tool falls back to cross-validating within that single year instead. Treat the reported MAE/RMSE as optimistic, and add another year of data if you can before trusting this fit for real use.

Either way, cross-check the printed Florida-weighted MAE against the current production model's (0.0179, per the *Model Review* document) — a large jump in either direction is worth a second look before promoting a new fit.

5 Reading the output

The model file

Whatever path you passed to `--output` (or `models/cc/lag3_ridge.json` by default) now holds the new fit: coefficients, feature means/standard-deviations for un-scaling, and the metrics/metadata it was fit with. The scoring tool reads this file directly — no other step is needed for the refreshed weights to take effect.

The history log

Every run appends one row to `results/cc/lag3_retrain_history.csv`, so accuracy can be tracked across refits over time:

COLUMN	WHAT IT TELLS YOU
<code>retrained_at_utc</code>	When this fit was run.
<code>mmg_files</code> , <code>alice_file</code>	Which source files were used.
<code>validation_method</code> , <code>train_years</code>	How the fit was validated, and which years it trained on.
<code>fl_weighted_mae</code> , <code>fl_weighted_rmse</code>	Florida accuracy — the number to watch run over run.
<code>output_model</code>	Which file this run wrote to.

6 If something goes wrong

MESSAGE	WHAT IT MEANS
<code>MMG workbook not found</code> / <code>ALICE CSV not found</code>	Check the file path is correct.
<code>Only N complete (year, year-3) transitions found</code>	The <code>--mmg</code> files you passed don't span enough years — you need data with both a target year and an observation 3 years earlier for the same ZIPs.
<code>Persisted model did NOT reproduce the in-memory fit</code>	The round-trip check failed — this should be rare. Re-run, and flag it to Tom or Lee if it happens again; the tool exits without leaving a broken model file behind.
Warning mentioning <code>openpyxl</code> or <code>numexpr</code>	Same fix as the scoring tool: <code>pip install --upgrade openpyxl numexpr</code> .

7 Who should run this

Unlike scoring, this isn't something to casually hand off — it overwrites the model every other user's forecasts depend on. Keep it to whoever maintains the repo (Tom, Lee, or a technical collaborator comfortable reviewing the printed accuracy numbers before trusting a new fit). A

bad refit is recoverable — `models/` is tracked in git, unlike `data/` — but it's still worth treating deliberately rather than running reflexively.

Confirmed: this tool fits *only* the 3-year lag, ridge specification — the one actually used for scoring. For the other lag specs, the plain-OLS variant, or a side-by-side comparison across all six, use `notebooks/1.6-cc-persisted-ridge-vs-ols-model.ipynb` instead.